BTS SIO — Option SLAM

Session 2026

DOCUMENTATION TECHNIQUE

Application Desktop de Statistiques

Ecole du Cinéma de Toulouse

Anaïs MARTINS
N° candidat : 02544722787
IRIS Mediaschool Toulouse

Épreuve E6 — Conception et développement d'applications (option SLAM)
Réalisation professionnelle n°2 — Client lourd

Sommaire

1. Présentation du projet
2. Stack technique et environnement
3. Architecture du projet
4. Communication avec l'API REST
5. Authentification et gestion de session
6. Interfaces utilisateur
7. Charte graphique JavaFX
8. Lancement et prérequis

1. Présentation du projet

1.1 Contexte et lien avec le Projet 1

Le Projet 2 est une application desktop (client lourd) développée en Java avec JavaFX. Elle constitue un complément au Projet 1 (application web) : plutôt que de gérer les données, elle les visualise sous forme de statistiques graphiques. Elle ne possède pas sa propre logique de base de données : elle s'appuie entièrement sur l'API REST du Projet 1, qui doit être lancé au préalable.

Les deux projets partagent la même base de données MySQL (centre_formations), les mêmes comptes utilisateurs et le même système d'authentification par JWT. Du point de vue de l'utilisateur, l'expérience est unifiée : les mêmes identifiants fonctionnent sur les deux applications.

1.2 Objectifs

L'application répond à un besoin de pilotage visuel : offrir à chaque profil d'utilisateur une vue synthétique de ses données sous forme de KPIs (indicateurs clés) et de graphiques interactifs. Elle est réalisée seule, de février à avril 2026, sous Windows 11 avec Visual Studio Code.

1.3 Les trois profils utilisateurs

Profil	Contenu du dashboard
Administrateur	KPIs globaux (nombre d'apprenants, formateurs, formations, taux de présence), graphique des inscrits par formation (BarChart), répartition des utilisateurs (PieChart), tableau des dernières inscriptions.
Formateur	KPIs personnels (matière enseignée, apprenants suivis, moyenne promotion), répartition des notes (PieChart), tableau de ses cours, tableau de ses apprenants avec moyenne, présence et avis.
Apprenant	KPIs personnels (formation suivie, moyenne générale, taux de présence), répartition de ses notes (PieChart), moyennes par matière (BarChart).

1.4 Comptes de test

Rôle	Identifiant	Mot de passe
Admin	anaismartins	mdp1
Formateur	merylstreep	mdp4
Apprenant	camilledupont	mdp2

2. Stack technique et environnement

2.1 Langage et plateforme

Java 25

Java est un langage de programmation orienté objet, compilé et fortement typé. Il est exécuté sur la JVM (Java Virtual Machine), ce qui le rend portable sur tous les systèmes d'exploitation. Dans ce projet, Java est le langage principal : il gère la logique métier, les appels HTTP vers l'API, le parsing JSON et l'interaction avec l'interface graphique.

JavaFX

JavaFX est le framework officiel de Java pour créer des interfaces graphiques desktop (fenêtres, boutons, tableaux, graphiques...). Il remplace l'ancien framework Swing. JavaFX propose un système de composants riches : labels, tableaux (TableView), graphiques (BarChart, PieChart), zones de saisie, etc. Il gère également le style visuel via des fichiers CSS JavaFX. Dans ce projet, toute l'interface — écran de connexion, dashboards, KPIs, graphiques — est construite avec JavaFX.

FXML

FXML est un langage basé sur XML qui permet de décrire la structure d'une interface JavaFX dans un fichier séparé, indépendamment du code Java. Plutôt que de créer les composants graphiques directement dans le code (ce qui mélange logique et présentation), on définit la disposition, les identifiants et les liaisons dans un fichier .fxml, et le contrôleur Java récupère les composants via l'annotation @FXML. Cela équivaut à la séparation HTML/JavaScript en web.

Gson 2.13.1

Gson est une bibliothèque Java développée par Google, qui permet de convertir du JSON en objets Java et vice versa. Quand l'API REST renvoie une réponse JSON (par exemple une liste d'apprenants), Gson la désérialise automatiquement en objets Java (List<Apprenant>) en faisant correspondre les clés JSON aux champs des classes modèle. C'est l'équivalent Java de JSON.parse() en JavaScript.

2.2 Communication réseau

HttpURLConnection

HttpURLConnection est la classe Java native (intégrée au JDK, sans dépendance externe) qui permet d'ouvrir des connexions HTTP/HTTPS et d'envoyer des requêtes GET, POST, etc. Dans ce projet, elle est utilisée dans ApiService.java pour interroger l'API REST du Projet 1. On l'instancie depuis une URL, on lui ajoute les en-têtes (dont le token JWT), on lit la réponse, et Gson se charge de la désérialiser.

SSLUtil — désactivation de la vérification SSL

Puisque l'API du Projet 1 tourne en HTTPS avec un certificat auto-signé (non reconnu par les autorités de certification), Java refuserait par défaut d'établir la connexion et lèverait une exception SSLHandshakeException. La classe utilitaire SSLUtil.java contourne ce comportement en remplacant le gestionnaire SSL par défaut par un gestionnaire permissif qui accepte tous les certificats. Cette désactivation est appelée une seule fois au démarrage de l'application, dans Main.java.

2.3 Outils de développement

Visual Studio Code

Environnement de développement utilisé pour écrire l'ensemble du code Java. Extensions utilisées : Extension Pack for Java (Microsoft), JavaFX support. L'application est lancée via l'onglet Run > Start Debugging de VSCode.

XAMPP et phpMyAdmin

Comme pour le Projet 1, XAMPP fournit le serveur MySQL local. La base de données centre_formations est partagée entre les deux projets. Le Projet 2 n'accède jamais directement à la base : il passe exclusivement par l'API REST du Projet 1.

GitHub

Le code source est versionné sur GitHub : github.com/Anais-Martins/Appli_Desktop-Ecole_du_cinema. La documentation en ligne est accessible sur anais-martins.github.io/projet-situation-pro-2.html.

3. Architecture du projet

3.1 Vue d'ensemble et pattern MVC

Le projet suit une architecture MVC (Modèle - Vue - Contrôleur), adaptée à JavaFX :

- Le Modèle (model/) représente les données métier (Apprenant, Formateur, Note, Cours...).
- La Vue (view/) est définie dans les fichiers FXML : elle décrit la structure visuelle de chaque écran.
- Le Contrôleur (controller/) contient la logique : il reçoit les événements utilisateur, appelle les services, et met à jour l'affichage.
- Les Services (service/) font le pont entre les contrôleurs et l'API REST : ils envoient les requêtes HTTP, reçoivent les réponses JSON et calculent les statistiques.

Le flux complet d'un écran est le suivant : le fichier FXML décrit la mise en page, le contrôleur associé est instancié automatiquement par JavaFX lors du chargement du FXML, la méthode initialize() est appelée automatiquement, elle déclenche les services, qui interrogent l'API, reçoivent du JSON, Gson le désérialise en objets Java, et le contrôleur injecte les résultats dans les composants visuels (Labels, TableView, Chart...).

3.2 Description des dossiers

src/app/view/

Contient tous les fichiers FXML qui définissent la structure visuelle de chaque écran : Login.fxml, DashboardAdmin.fxml, DashboardFormateur.fxml, DashboardApprenant.fxml. Chaque fichier déclare le contrôleur associé via l'attribut fx:controller, et donne des identifiants (fx:id) aux composants que le contrôleur devra manipuler.

src/app/controller/

Contient un contrôleur par vue : LoginController.java, DashboardAdminController.java, DashboardFormateurController.java, DashboardApprenantController.java. Chaque contrôleur déclare les composants injectés via @FXML, instancie son service de statistiques, et implémente la méthode initialize() qui déclenche le chargement des données.

src/app/model/

Contient les classes Java qui représentent les entités métier : Apprenant.java, Formateur.java, Note.java, Cours.java, Presence.java, Inscription.java, Formation.java, User.java, LoginResponse.java, ApprenantFormateur.java... Ces classes servent à Gson pour désérialiser les réponses JSON de l'API : les noms de champs Java doivent correspondre exactement aux clés JSON renvoyées. Elles contiennent uniquement des champs privés, des getters et des setters.

src/app/service/

Contient les classes qui effectuent les appels à l'API REST et calculent les statistiques : ApiService.java (service HTTP générique réutilisé par tous), StatAdminService.java, StatFormateurService.java, StatApprenantService.java, UserService.java (pour la connexion). Chaque service de statistiques reçoit en constructeur l'identifiant de l'utilisateur connecté et expose des méthodes comme getNbApprenants(), getMoyennePromotion(), getRepartitionNotes()...

src/app/util/

Contient les classes utilitaires. SessionUser.java est une classe statique qui stocke en mémoire l'utilisateur connecté et son token JWT pour toute la durée de la session. SSLUtil.java désactive la vérification du certificat SSL pour permettre la communication avec l'API locale en HTTPS auto-signé.

src/app/Main.java

Point d'entrée de l'application. Extend Application (classe JavaFX), appelle SSLUtil.disableSSLVerification() au démarrage pour gérer le certificat auto-signé, charge le fichier Login.fxml via FXMLLoader, crée la fenêtre principale (Stage) et l'affiche.

4. Communication avec l'API REST

4.1 Principe général

L'application desktop ne possède aucune connexion directe à la base de données MySQL. Toutes les données sont obtenues en interrogeant l'API REST du Projet 1, qui tourne sur https://localhost:3001. Cette architecture présente plusieurs avantages : la logique métier est centralisée dans l'API, les deux applications restent indépendantes, et l'API peut être utilisée par d'autres clients à l'avenir (application mobile, etc.).

4.2 ApiService.java — le service HTTP générique

ApiService est une classe réutilisable qui centralise toute la logique des appels HTTP. Elle définit l'URL de base (https://localhost:3001) et propose une méthode générique get() qui :

- Ouvre une connexion HTTP vers l'endpoint demandé via HttpURLConnection.
- Ajoute automatiquement le token JWT dans l'en-tête Authorization: Bearer <token>, récupéré depuis SessionUser.getToken().
- Lit la réponse JSON depuis le flux d'entrée (InputStream).
- Désérialise le JSON en objet Java du type demandé via Gson.
- Retourne null en cas d'erreur HTTP ou d'exception, avec un message dans la console.

La méthode get() est surchargée en deux versions :

- get(endpoint, Class<T> classType) : pour désérialiser en un objet simple.
- get(endpoint, Type type) : pour désérialiser en type générique (List<Note>, List<Cours>...) via TypeToken de Gson.

Exemple d'utilisation dans un service :

```
ApiService api = new ApiService();
// Objet simple
Apprenant a = api.get("/api/apprenant/5", Apprenant.class);
// Liste
Type listType = new TypeToken<List<Note>>(){}.getType();
List<Note> notes = api.get("/api/apprenant/5/notes", listType);
```

4.3 Gestion des réponses paginées

Certains endpoints de l'API Node.js renvoient les données dans un objet enveloppé (wrapé) plutôt que directement en tableau. Par exemple, GET /api/apprenant renvoie :

```
{ "apprenants": [...], "pagination": { "page": 1, "totalPages": 3 } }
```

Pour gérer ces cas, ApiService définit des classes internes statiques (ApprenantResponse, FormateurResponse, InscriptionResponse...) qui correspondent à la structure de la réponse. Gson les désérialise, puis le service extrait la liste via le getter correspondant.

4.4 Contournement SSL

Avant tout appel HTTP, Main.java appelle SSLUtil.disableSSLVerification(). Cette méthode installe un TrustManager Java personnalisé qui accepte tous les certificats sans vérification, et un HostnameVerifier qui accepte tous les noms d'hôte. Sans cette étape, Java refuserait la connexion HTTPS car le certificat auto-signé du Projet 1 n'est pas reconnu par le magasin de certificats Java.

4.5 Cache local dans les services

Pour éviter de multiplier les appels HTTP identiques lors de l'initialisation d'un dashboard, les services de statistiques (StatFormateurService, StatAdminService, StatApprenantService) implémentent un cache local simple : les listes récupérées (cours, notes, apprenants...) sont stockées dans des attributs privés. Si la méthode est appelée plusieurs fois, les données sont renvoyées depuis le cache sans nouvel appel HTTP.

```
public List<Cours> getCours() {
    if (coursList != null) return coursList; // cache
    coursList = api.get("/api/formateur/" + idFormateur + "/cours",
listType);
    return coursList;
}
```

5. Authentification et gestion de session

5.1 Flux de connexion

L'écran de connexion (Login.fxml) propose deux champs : identifiant et mot de passe. Lorsque l'utilisateur clique sur « Se connecter », la méthode handleLogin() du LoginController est déclenchée. Elle appelle UserService.login(username, password), qui envoie une requête POST à l'endpoint /api/login de l'API REST du Projet 1 avec les identifiants en JSON. L'API vérifie les identifiants en base de données et, si corrects, renvoie un objet JSON contenant :

- success : true ou false
- token : le token JWT généré par l'API
- user : objet contenant id_user, username, role, id_apprenant

Gson désérialise cette réponse en un objet LoginResponse.java. Si la connexion réussit, les informations sont sauvegardées dans SessionUser, puis l'utilisateur est redirigé vers le dashboard correspondant à son rôle.

5.2 SessionUser — gestion de la session

SessionUser est une classe utilitaire à méthodes statiques qui joue le rôle de « mémoire de la session » pendant toute la durée de l'exécution de l'application. Elle stocke deux informations :

- currentUser : l'objet User contenant les informations de l'utilisateur connecté (username, role, id_apprenant, id_formateur).
- token : le token JWT à inclure dans chaque requête API.

Elle expose des helpers statiques : getUsername(), getRole(), getIdApprenant(), getIdFormateur(), getToken(). La méthode clear() vide la session lors de la déconnexion. Comme elle est statique, toutes les classes du projet y accèdent sans instanciation.

> *Différence avec le Projet 1 : dans l'application web, le token est stocké dans un cookie du navigateur. Dans l'application desktop, il n'y a pas de navigateur, donc le token est stocké directement en mémoire dans SessionUser, et injecté manuellement dans chaque requête HTTP via l'en-tête Authorization.*

5.3 Redirection selon le rôle

Après connexion réussie, le LoginController appelle la méthode ouvrirDashboard() avec le chemin du fichier FXML correspondant au rôle :

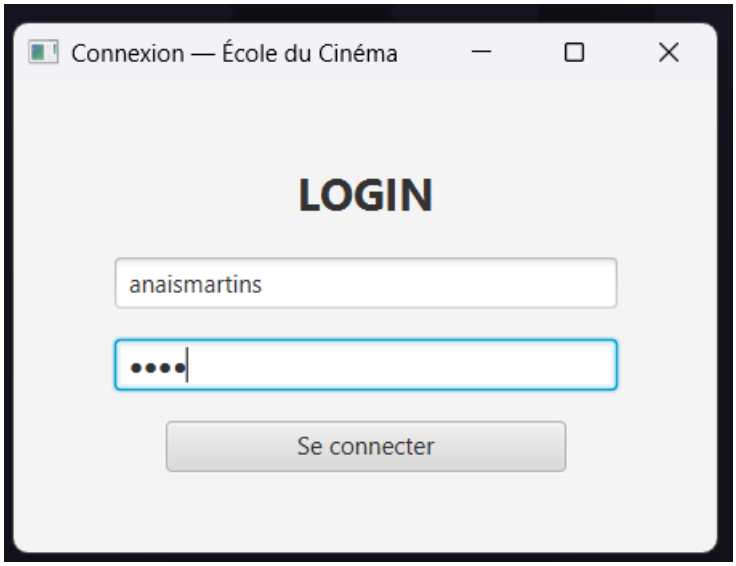
- admin → /app/view/DashboardAdmin.fxml
- formateur → /app/view/DashboardFormateur.fxml
- apprenant → /app/view/DashboardApprenant.fxml

Un nouveau Stage (fenêtre JavaFX) est ouvert avec le dashboard chargé, et la fenêtre de connexion est masquée. La déconnexion effectue l'inverse : elle appelle SessionUser.clear(), charge à nouveau Login.fxml dans un nouveau Stage, et ferme le dashboard.

6. Interfaces utilisateur

6.1 Fenêtre de connexion

L'écran de connexion est la première fenêtre affichée au lancement. Il est défini dans Login.fxml et contrôlé par LoginController.java. Sa structure est minimaliste : un VBox centré avec un titre « LOGIN », un champ texte (TextField) pour l'identifiant, un champ mot de passe (PasswordField) qui masque la saisie, et un bouton « Se connecter ». Le bouton est lié à la méthode handleLogin() via l'attribut FXML onAction. En cas d'identifiants incorrects ou de champs vides, une alerte JavaFX (Alert.AlertType.INFORMATION) est affichée.



6.2 Dashboard Administrateur

Le dashboard admin (DashboardAdmin.fxml / DashboardAdminController.java) est la vue la plus complète. Il est organisé en plusieurs zones :

KPIs

Une rangée de quatre encarts affichant en grand les indicateurs clés : nombre d'apprenants, nombre de formateurs, nombre de formations, et taux de présence global. Ces valeurs sont calculées par StatAdminService et injectées dans des Labels JavaFX via kpiApprenants.setText(). Le taux de présence est formaté en pourcentage avec String.format("%.0f%%", taux).

BarChart — inscrits par formation

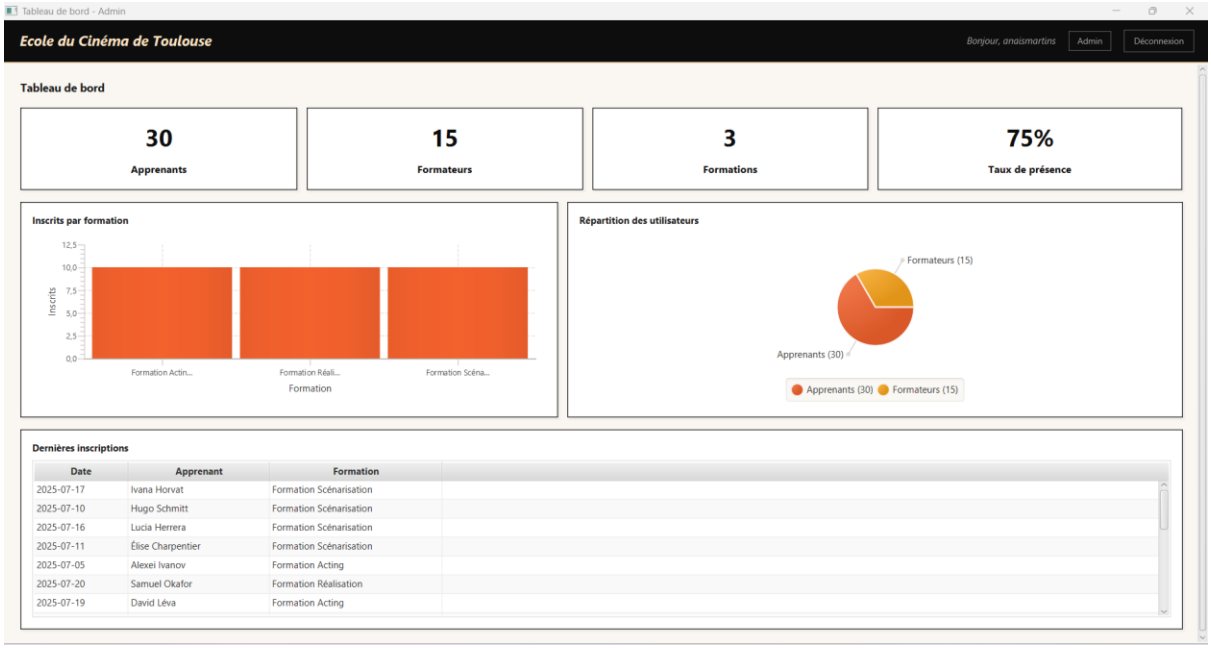
Un graphique en barres (BarChart<String, Number>) affiche le nombre d'inscrits pour chacune des trois formations. Les données sont récupérées via StatAdminService.getInscriptionsParFormation(), qui renvoie une liste de tableaux Object[] où chaque élément contient le nom de la formation et le nombre d'inscrits. Les noms trop longs sont tronqués à 15 caractères pour l'axe X.

PieChart — répartition des utilisateurs

Un graphique circulaire (PieChart) affiche la répartition entre apprenants et formateurs. Les données sont construites directement à partir des KPIs déjà chargés (nombre d'apprenants et de formateurs), en créant des PieChart.Data avec le nom et la valeur.

Tableau des dernières inscriptions

Un TableView à trois colonnes (Date, Apprenant, Formation) affiche les 20 dernières inscriptions triées par date décroissante. JavaFX TableView utilise une classe interne InscriptionRow avec des getters dont le nom correspond aux colonnes, liaison faite via PropertyValueFactory.



6.3 Dashboard Formateur

Le dashboard formateur (DashboardFormateur.fxml / DashboardFormateurController.java) est personnalisé pour chaque formateur connecté. L'identifiant du formateur est récupéré depuis SessionUser.getIdFormateur() et passé au constructeur de StatFormateurService.

KPIs

Trois encarts affichent : la matière enseignée (nom en grand, récupéré depuis le premier cours du formateur), le nombre d'apprenants suivis, et la moyenne de la promotion (calculée sur l'ensemble des notes données par le formateur, formatée avec une décimale).

PieChart — répartition des notes

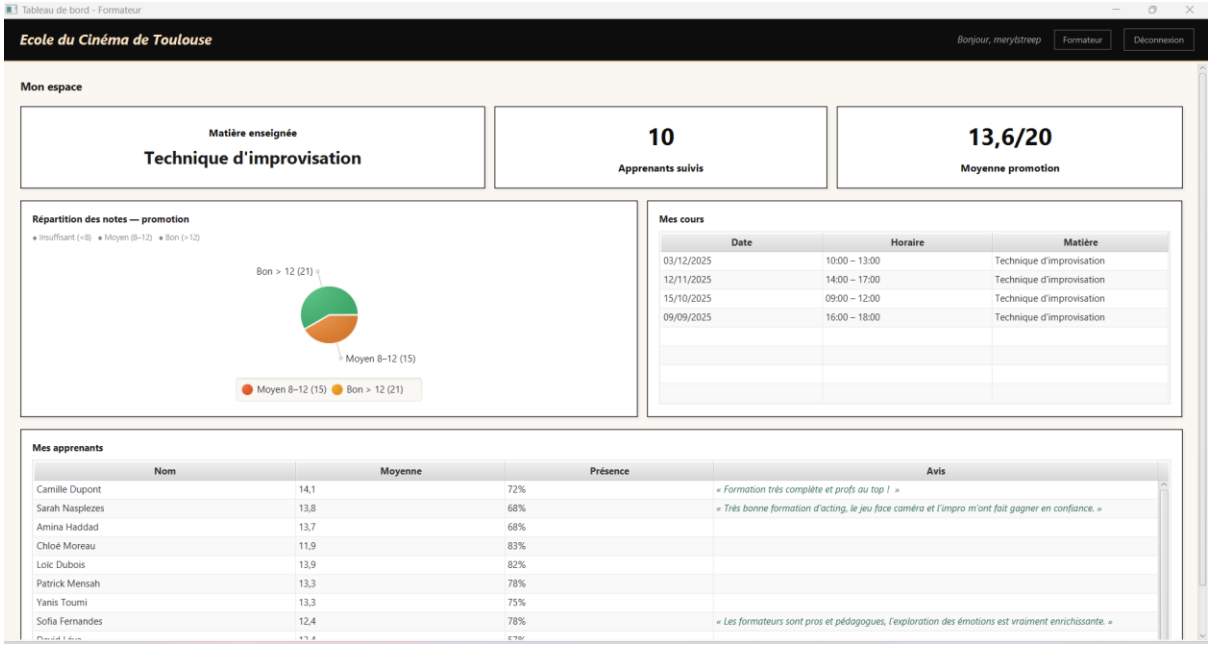
Un PieChart découpe les notes de la promotion en trois catégories : Insuffisant (< 8), Moyen (8 à 12), Bon (> 12). Le calcul est effectué par StatFormateurService.getRepartitionNotesPromotion(), qui retourne un tableau de trois entiers.

Tableau Mes cours

Un TableView liste les cours du formateur avec la date, l'horaire (début – fin) et la matière. Les cours sont récupérés via GET /api/formateur/:id/cours.

Tableau Mes apprenants

Un TableView liste les apprenants du formateur avec leur nom complet, leur moyenne (calculée via getMoyenneApprenant()), leur taux de présence (getTauxPresenceApprenant()), et leur avis s'il en ont déposé un. Les moyennes et taux de présence sont recalculés à partir des endpoints de l'apprenant pour garantir la cohérence avec le dashboard apprenant.



6.4 Dashboard Apprenant

Le dashboard apprenant (DashboardApprenant.fxml / DashboardApprenantController.java) est personnalisé pour l'apprenant connecté, dont l'identifiant est récupéré via SessionUser.getIdApprenant().

KPIs

Trois encarts affichent : la formation suivie (nom récupéré via GET /api/apprenant/:id/formations), la moyenne générale (calculée sur toutes les notes de l'apprenant, affichée sur 20), et le taux de présence personnel (rapport entre présences true et nombre total de présences enregistrées).

PieChart — répartition des notes

Même logique que pour le formateur, mais appliquée uniquement aux notes de l'apprenant connecté : Insuffisant (< 8), Moyen (8 à 12), Bon (> 12).

BarChart — moyennes par matière

Un BarChart affiche la moyenne de l'apprenant pour chaque matière qu'il a suivie. Le calcul est effectué en regroupant les notes par id_matiere, en calculant la somme et en divisant par le nombre de notes par matière.

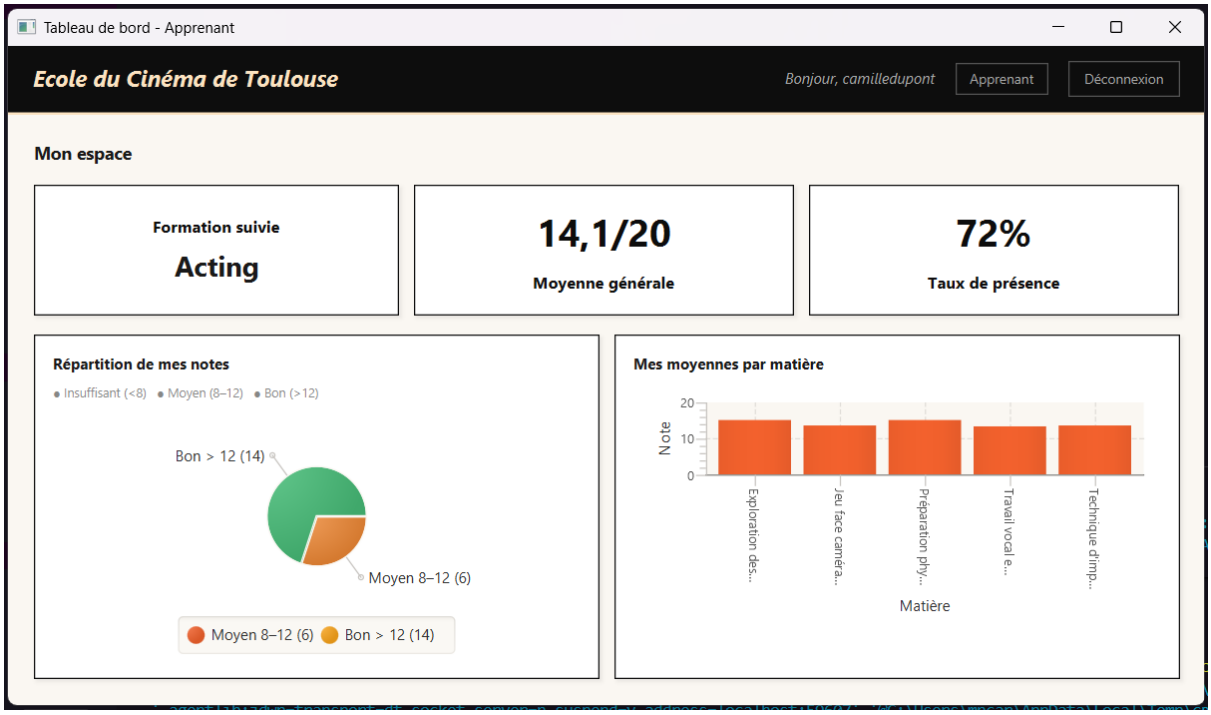


Tableau de bord - Apprenant
Ecole du Cinéma de Toulouse
Bonjour, camilledupont
Apprenant
Déconnexion
Mon espace
Formation suivie
Acting
14,1/20
Moyenne générale
72%
Taux de présence
Répartition de mes notes
Insuffisant (<8)
Moyen (8–12)
Bon (>12)
Bon > 12 (14)
Moyen 8–12 (6)
Moyen 8–12 (6)
Bon > 12 (14)
Mes moyennes par matière
Note
20
10
0
Exploration des...
Jeu face caméra...
Préparation phy...
Travail vocal e...
Technique d'imp...
Matière

7. Charte graphique JavaFX

La charte graphique du Projet 2 est cohérente avec celle du Projet 1 : mêmes couleurs (noir #0d0d0d, beige #FCE4C2, fond #FAF7F2), même style minimaliste sans border-radius ni couleurs vives. Les styles sont définis directement dans les attributs style des fichiers FXML, sous forme de propriétés CSS JavaFX (qui utilisent le préfixe -fx-).

Élément	Style appliqué
Fond général (VBox racine)	-fx-background-color: #FAF7F2
Header	-fx-background-color: #0d0d0d avec bordure basse #FCE4C2 (2px)
Titre header	-fx-text-fill: #FCE4C2 ; -fx-font-style: italic
Nom utilisateur	-fx-text-fill: #aaaaaa ; -fx-font-style: italic
Badge rôle	Bordure #555555, fond transparent, texte #aaaaaa
Encarts KPI	Fond blanc, ombre : dropshadow(gaussian, rgba(0,0,0,0.06), 4, 0, 2, 2)
Titre KPI	-fx-font-weight: bold
Valeur KPI	Grande taille de police (-fx-font-size: 28px ou plus)

Les graphiques JavaFX (BarChart, PieChart) utilisent les couleurs par défaut de JavaFX pour les séries de données, avec le fond du graphique mis à transparent pour s'intégrer dans le fond beige général. La légende du BarChart des inscrits est masquée (setLegendVisible(false)) pour alléger la présentation.

8. Lancement et prérequis

8.1 Prérequis

- Java 25 installé sur la machine.
- XAMPP lancé avec le module MySQL actif.
- Le Projet 1 (application web) doit être lancé en premier via npm run dev dans son répertoire. L'API REST doit être accessible sur https://localhost:3001.
- Les bibliothèques JavaFX et Gson 2.13.1 configurées dans le projet.

8.2 Lancement

L'application se lance depuis Visual Studio Code :

- Ouvrir le dossier du projet dans VSCode.
- Aller dans l'onglet Run (Executer) de la barre latérale.
- Cliquer sur Start Debugging (ou appuyer sur F5).
- La fenêtre de connexion s'ouvre.

> *Ordre impératif : lancer d'abord le Projet 1 (npm run dev), puis le Projet 2. Si l'API n'est pas disponible, toutes les requêtes HTTP retourneront null et les dashboards s'afficheront vides.*

8.3 Accès aux ressources

- Code source : https://github.com/Anais-Martins/Appli_Desktop-Ecole_du_cinema
- Documentation en ligne : https://anais-martins.github.io/projet-situation-pro-2.html
- Archive ZIP téléchargeable sur la page de documentation ci-dessus